INT 6011 – Image Processing and Video Analysis

Final Exam Assignment

Video Compression

Le Cong Thuong
thuonglc@vnu.edu.vn

January 19, 2026

Overview

In digital signal processing and multimedia, the efficient representation of visual sequences is fundamental. This assignment explores the core principles of video compression, bridging the gap between massive raw pixel data and the constrained bandwidth of the internet. Why does this matter? Applications like high-definition streaming (Netflix, YouTube), real-time video conferencing (Zoom), and cloud gaming rely on massive compression ratios to function. Without efficient video coding, a mere few seconds of uncompressed 4K video would consume gigabytes of data, rendering modern digital communication impossible.

Think of video compression as a game of "predictive elimination": rather than storing every pixel, we aim to transmit only the "surprise" the information that cannot be guessed from what has already been seen. We'll model **Intra-frame prediction** to exploit spatial smoothness within an image, extend this to **Inter-frame prediction** to exploit temporal coherence across time, and analyze the resulting estimated rate-distortion trade-offs. Building on these classical foundations, we'll explore deep learning models like RAFT [1], which predict dense motion fields end-to-end. This progression moves from rigid block-based engineering to continuous learning paradigms, echoing the field's shift toward Neural Video Codecs.

Learning goals. By the end of this assignment, you should be able to:

- Implement the core DPCM codec loop (Prediction → Residual → Quantization → Reconstruction) for Intra-frame compression. From that, explain the relationship between Quantization step size (Q), Distortion (PSNR), and Bitrate (Entropy).
- Implement Block Matching to exploit temporal redundancy between consecutive video frames.
- Analyze the efficiency of Spatial vs. Temporal prediction and identify "blocking artifacts" in reconstructed images.
- Explore data-driven motion estimation using a deep model like RAFT [1], comparing the trade-offs between sparse block vectors and dense optical flow fields.

Starter code and environment setup

Starter code. Use the provided Python starter code in `code_starter/`.

Environment setup. Use Python $\geq$ **3.8** and install the dependencies from `requirements.txt`:

What to submit.

- Your code: `code_starter/*.py`.
- A short PDF report answering the questions and including the required figures.

Starter code, data and environment setup

Starter code. Use the provided Python starter code in `code_starter/`.
Data. Images used in this assignment are from the open-source short film *Big Buck Bunny*[1], Foreman sequence [2]
Environment setup. Use Python $\geq$ **3.8** and install the dependencies from `requirements.txt`:

1 The "Mini-Codec": Intra-frame prediction & Quantization

Raw 1080p video at 60 fps requires bandwidth $1920 \times 1080 \times 60$ fps $\times$ 24 bits/pixel $\approx$ 3 Gbit/s, making it impractical for streaming or storage without compression. Modern video codecs, such as H.264/AVC [2], follow a key philosophy: Don't store what you can predict. Instead of saving every pixel, codecs predict pixel values based on spatial (within a frame) or temporal (across frames) redundancies and only encode the differences (residuals). In this section, we focus on intra-frame redundancy (spatial compression within a single image frame). Neighboring pixels in an image are often similar, so we can predict a block's content using pixels from adjacent, already-processed blocks. You'll build a "Mini-Codec" that processes an image in 16x16 blocks, implementing the Prediction Model and Transform/Quantize blocks from the high-level H.264 encoder flowchart (see Figure 1).

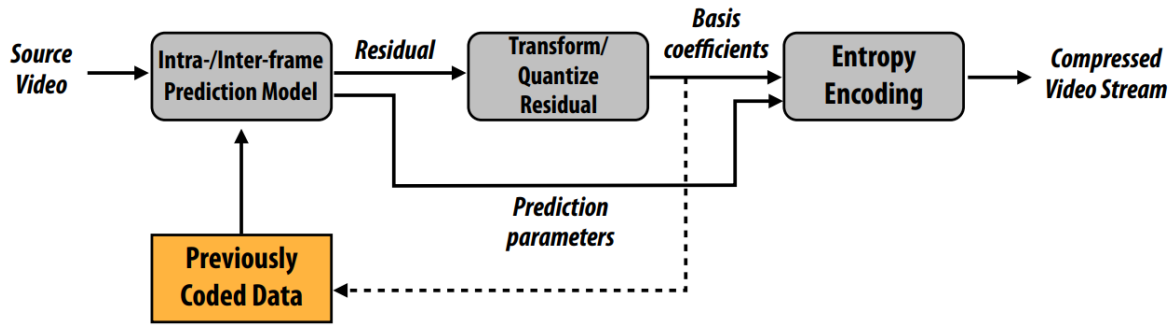


Figure 1: **The Hybrid Video Encoder Pipeline.** In Part 1, you will implement the components highlighted in the Prediction Model (Intra) and the Quantization block. Note the feedback loop: the prediction relies on the *reconstructed* image, not the source.

Your Mini-codec must iterate over the image in raster scan order (left-to-right, top-to-bottom). For every 16×16 block, perform the following DPCM (Differential Pulse-Code Modulation) cycle:

1. **Predict (P):** Generate a prediction block using the pixels from the *already reconstructed* neighbors (top row and left column).

[1] `http://commondatastorage.googleapis.com/gtv-videos-bucket/sample/BigBuckBunny.mp4`
[2] `https://hlevkin.com/hlevkin/TestVideo/foreman.yuv`

2. **Residual (R):** Calculate the prediction error relative to the source image (I_{src}):

$$R = I_{src} - P$$

3. **Quantize (R_q):** Reduce the precision of the residual to save bits. This is the lossy step:

$$R_{quant} = \text{round}\left(\frac{R}{Q}\right) \times Q$$

where Q is the quantization step size.

4. **Reconstruct (I_{rec}):** Simulate the decoder's view. Since the decoder only receives R_q and the prediction rule, it reconstructs the image as:

$$I_{rec} = P + R_{quant}$$

5. **Update State:** Overwrite the current block in your image buffer with I_{rec}. The *next* block (to the right or below) must use these reconstructed values for its prediction to ensure the encoder and decoder stay in sync.

1.1 Work to do

You are given images in `/data/part_1/`, and you must complete the code in `part1_codec.py`. You may also use the `get_frames.py` to download, extracts additional images if needed.

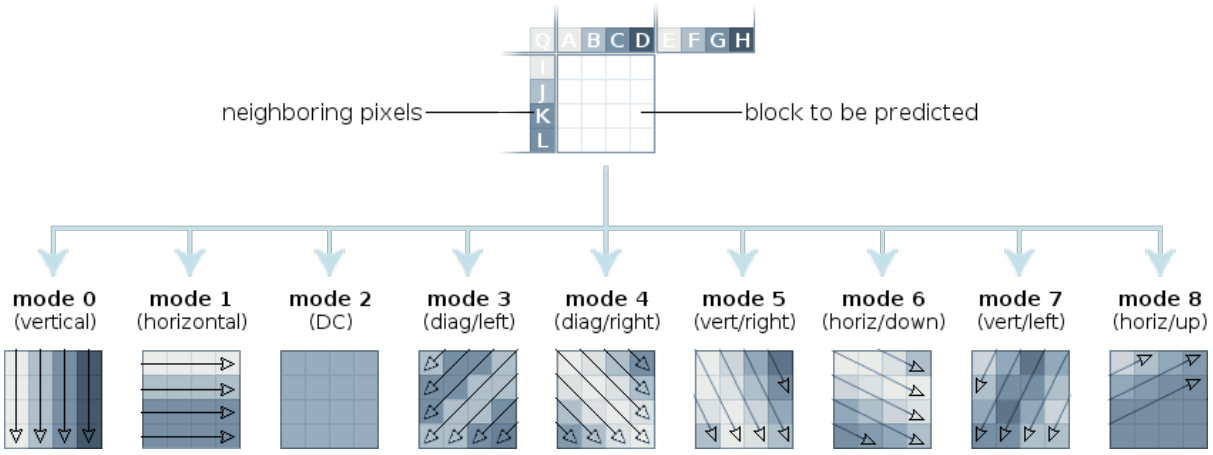


Figure 2: **Intra-Prediction Modes.** The grid represents the current 4×4 block. **Vertical Mode (mode 0):** Extrapolates the boundary pixels of the Top neighbor downwards. **Horizontal Mode (mode 1):** Extrapolates the boundary pixels of the Left neighbor across.

1. **Prediction Logic (`get_prediction`):** Implement the helper method to generate prediction blocks using *reconstructed* neighbors from `recon_image`.
   - Handle **Vertical** (row above) and **Horizontal** (column left) modes.
   - Manage boundary conditions (edges) and the top-left corner case (default to 128).
2. **DPCM Encoding Loop (`encode_image`):** Iterate over image blocks and implement the full DPCM cycle.
3. **Metrics (`calculate_metrics`):** Compute the final performance stats:
   - **PSNR:** Measure distortion between I_{src} and I_{rec}.

- **Bitrate:** Calculate Shannon Entropy of the quantized residuals (R_q) to estimate BPP. This estimates the theoretical lower bound of bits per pixel (BPP):

$$H(X) = -\sum_i P(x_i) \log_2 P(x_i)$$

1.2 Report Questions

- **Mode Statistics:** Across the entire image, what percentage of blocks preferred Vertical Mode vs. Horizontal Mode?
- **Quantization–Distortion with Estimated Rate:** Run the codec with $Q \in \{1, 5, 10, 15, 20, 25, 30, 40, 50\}$. Plot a curve with **Estimated bitrate (entropy, bpp)** on the x-axis and **PSNR (dB)** on the y-axis. Discuss the trade-off.
- **Artifact Analysis:** Display the reconstructed image at $Q = 50$ and zoom in on an area with sharp edges. Analyze the origin of the observed "Blocking Artifacts". Explain the relationship between prediction error and quantization: Why do high-frequency features (like edges) generate large residuals, and how does coarse quantization transform these residuals into visible discontinuities at the block boundaries?

2 Inter-frame Prediction: Block Matching

While Intra-frame prediction exploits spatial redundancy within a single frame, video is a sequence of images where consecutive frames are often highly correlated. Pixels in the current frame can frequently be described as a translation of pixels from a temporally adjacent frame (e.g., an object shifting slightly to the right). This correlation represents **temporal redundancy**.

In this section, we transition from Intra-frame prediction to Inter-frame prediction. You will implement a block-based **motion estimation** algorithm to identify the best match for a 16×16 block in a previously decoded *reference frame*, and apply **motion compensation** to generate the prediction (Fig. 3).

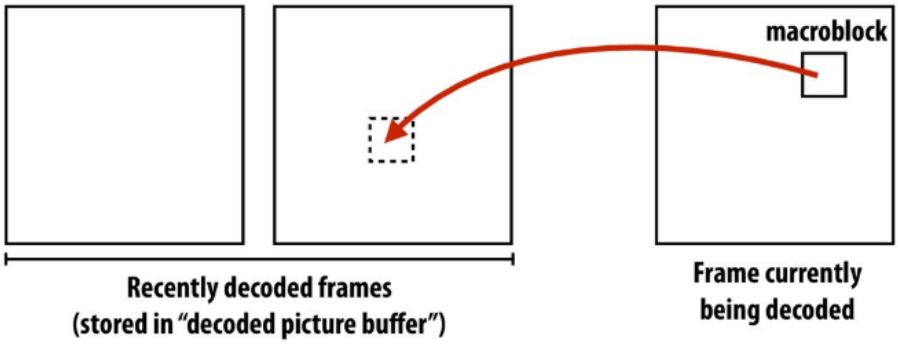


Figure 3: Motion compensation process: The prediction block is derived from the reference frame using the motion vector.

For every 16×16 macroblock in the current frame (I_{curr}), the encoding process is as follows:

1. **Motion Estimation (Search):** Instead of looking at immediate neighbors, the encoder searches a defined window in the *Reference Frame* (I_{ref}) to find the 16×16 block that most closely matches the current block. This spatial displacement is stored as a **Motion Vector** ($MV = (dx, dy)$).

2. **Motion Compensation (Predict):** The block P is simply the block of pixels fetched from the reference frame at the offset defined by the motion vector:

$$P(x, y) = I_{ref}(x + dx, y + dy)$$

3. **Residual (R):** Calculate the difference between the source and the motion-compensated prediction:

$$R = I_{curr} - P$$

4. **Quantize & Reconstruct:** Similar to Part 1, the residual is quantized, then added back to the prediction to form the reconstructed block for the next frame's reference.

2.1 Work to do

You started with images in `/data/part_2/`, and must implement code in `part2_motion.py`.

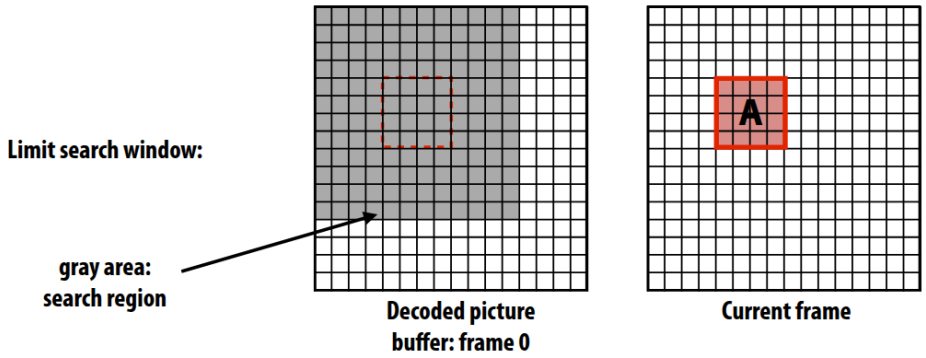


Figure 4: **Block Matching Motion Estimation.** The encoder searches the Reference Frame to find the best match (Motion Vector) for each block.

1. **Motion Estimation (`motion_estimation`):** Implement Full Search Block Matching. For every block in the current frame, search the defined window in the reference frame to identify the candidate with the minimum SAD. Return the optimal **Motion Vectors** (MV).

2. **Motion Compensation (`motion_compensation`):** Implement the decoder logic to reconstruct the predicted frame (P) using only the reference frame and the motion vectors:

$$P(y, x) = I_{ref}(y + dy, x + dx)$$

3. **Integration (`calc_residual`):** Combine the modules to simulate the full codec loop:
   - Call `motion_estimation` to generate vectors.
   - Call `motion_compensation` to generate the prediction P.
   - Compute the residual $R = I_{curr} - P$.

2.2 Report Questions

1. **Intra vs. Inter Efficiency (Experimental Comparison):** Compare the compression efficiency of spatial (Intra) versus temporal (Inter) prediction using the provided `curr_img` and `ref_img`.

- **Measure:** Run your code from Part 1 (Intra) and Part 2 (Inter) on `curr_img`. Report the **Residual Entropy (bpp)** for both methods.
- **Interpret:** Why is the entropy lower for one method? Discuss what type of redundancy (Spatial vs. Temporal) is more dominant in this specific image pair.

2. **Failure Cases of Inter-Prediction:** Inter-frame prediction relies on the assumption that blocks in the current frame exist in the reference frame.
   - **Experiment:** Use the `get_frames` tool to extract a pair of frames from a video scene cut, a fast pan, or a sudden lighting change.
   - **Result:** Run Motion Estimation on this new pair. Display the **Residual Image** and report its **Entropy**.
   - **Analysis:** Explain why Inter-prediction performs poorly in this specific case compared to the standard result in Q1. Include your chosen images in the report to support your explanation.
3. **The Cost of Searching (Parameter Sweep):** Motion estimation is the computational bottleneck of video encoding. The search range p determines the trade-off between compression quality and encoding speed.
   - **Experiment:** Run your motion estimation loop with search ranges $p \in \{4, 8, 16, 32, 64\}$.
   - **Plot:** Generate a dual-axis graph:
     - **X-axis:** Search Range p.
     - **Left Y-axis:** Execution Time (seconds).
     - **Right Y-axis:** Residual Entropy (bpp).
   - **Analysis:** Discuss the trade-off between 3 above parameters.

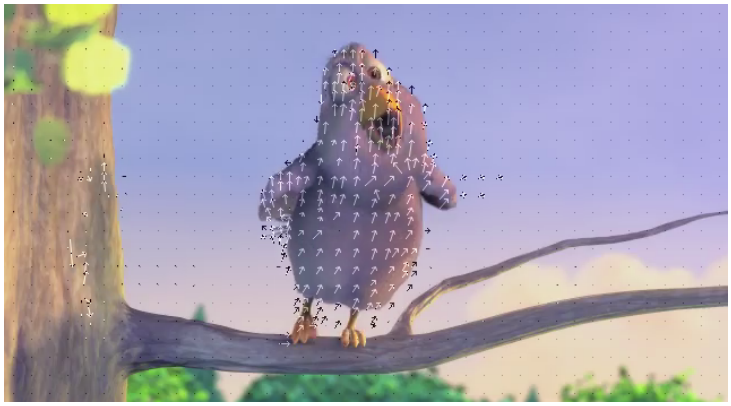
3 Inter-frame Prediction: Learned Motion Estimation

In Part 2, you implemented Block Matching for inter-frame prediction. While effective, it suffers from two major limitations:

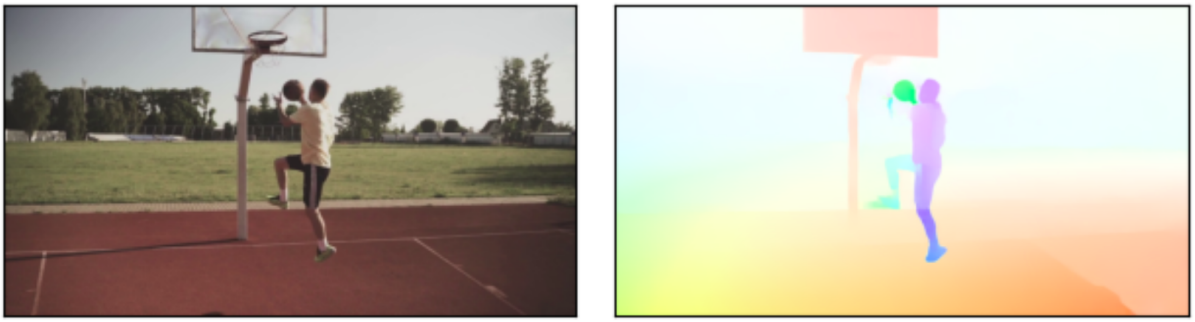
- **Blockiness:** Real-world objects do not move as rigid 16×16 tiles, creating visible seams at block boundaries in the prediction.
- **Quantization:** Motion is restricted to an integer grid, failing to capture sub-pixel movements or complex deformations (zoom, rotation).

To address these issues, we explore **Dense Optical Flow**. Instead of assigning one vector per block, we estimate a motion vector for *every single pixel*. In this section, you will use a deep neural network, **RAFT** (RAFT: Recurrent All-Pairs Field Transforms for Optical Flow) [1], to generate high-fidelity motion fields. The Deep Inter-frame prediction pipeline replaces the "Search" step in the Block matching with a Neural Network inference:

1. **Inference (Flow Estimation):** The CNN takes two frames (I_{ref}, I_{curr}) and outputs a Desne Optical Flow $F \in \mathbb{R}^{H \times W \times 2}$, where $F(x, y) = (\Delta x, \Delta y)$ represents the floating-point displacement for that pixel.
2. **Warping (Motion Compensation):** Since Δx is continuous, we cannot simply copy array indices. We must "warp" the reference frame using **Bilinear Interpolation**:
$$P(x, y) = I_{ref}(x + \Delta x, y + \Delta y)_{\text{interpolated}}$$
3. **Residual (R):** As before, we compute the difference to be encoded:
$$R = I_{curr} - P$$



(a) **Block Matching (Sparse):** One vector per 16×16 patch.



(b) **Optical Flow (Dense):** One vector per pixel.

Figure 5: Comparison of Motion Representations.

3.1 Work to do

You started with images in `/data/part_3/`, and must implement code in `part3_flow.py`.

1. **Load Pre-trained Model (`get_optical_flow_model`):** Use `torchvision.models.optical_flow` to load the **RAFT_SMALL** model with pre-trained weights. This model has learned to estimate motion from millions of synthetic and real video pairs.
2. **Implement Differentiable Warping (`warp_flow`):** The model gives you the vectors, but you must reconstruct the predicted image P using **Bilinear Interpolation**.
3. **Integration (`predict`):** Combine the RAFT flow estimation and `warp_flow` to generate the prediction P_{warp}, then compute the residual $R = I_{curr} - P_{warp}$.

3.2 Report Questions

1. **Qualitative Analysis (Visual Comparison):** Compare the visual quality of the predictions from Block Matching (Part 2) vs. Dense Optical Flow (Part 3).
   - **Prediction P Artifacts:** Zoom in on moving edges (facial boundary). Contrast the "blocking artifacts" of the block matching method with the "warping" behavior of Dense Optical Flow. Which appears more natural?
   - **Residual R "Ghosting":** Ideally, a residual should contain only random noise. **Question:** Compare the two Residual images. Which one shows "grid lines" and which one shows"ghosting" of edges? Give the reason for these symptoms.
2. **Quantitative Analysis (The Entropy Paradox):** Compute the **Residual Entropy (bpp)** for the Optical Flow method (H_{flow}) and compare it to your Block Matching result (H_{block}).

- **The Result:** You will likely find that the entropy for Optical Flow is **higher** (worse) than Block Matching, despite the prediction looking visually smoother.
- **Analysis:** Explain this counter-intuitive result.
  Hint: Consider the difference between **Copying** (Block Matching) and **Interpolating/Warping** (Optical Flow).

3. **The Cost of "Side Information":** A video codec must transmit both the Residual *and* the Motion Vectors.
   - **Calculation:** Calculate the uncompressed size (in bits) required to store the motion data for a single 355×288 frame:
     - **Block Matching:** One pair of 8-bit integers (dx, dy) per 16×16 block.
     - **Optical Flow:** One pair of 32-bit floats (u, v) per pixel.
   - **Discussion:** Compare these two numbers. Is it feasible to transmit raw Dense Optical Flow over the internet? What additional processing step would be strictly necessary to make Neural Video Coding practical?

Statement on AI Usage

Please include a brief section at the end of your report detailing how you collaborated with AI assistants during this assignment. Be transparent about whether they were used for debugging, code generation, or concept explanation.

References

[1] Z. Teed and J. Deng, "Raft: Recurrent all-pairs field transforms for optical flow," in *European conference on computer vision*. Springer, 2020, pp. 402–419.

[2] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the h. 264/avc video coding standard," *IEEE Transactions on circuits and systems for video technology*, vol. 13, no. 7, pp. 560–576, 2003.